EXPERIMENT 6 – ERROR DETECTION AND CORRECTION USING HAMMING CODE

AIM

To develop a program that applies Hamming Code for detecting and correcting transmission errors, and to verify its single-bit error correction capability.

ALGORITHM

Sender

1. Read a text message from the user.
2. Convert each character into an 8-bit binary value.
3. Divide the binary data into groups of four bits.
4. Add parity bits using Hamming (7,4) coding.
5. Save the encoded data in the file `channel.txt`.

Receiver

1. Read the encoded data from `channel.txt`.
2. Split the encoded stream into 7-bit codewords.
3. Calculate the parity checks and determine the syndrome.
4. Use the syndrome to identify the position of a possible error.
5. Correct the error by flipping the identified bit.
6. Remove the redundant parity bits.
7. Convert the remaining binary data into 8-bit characters.
8. Display the reconstructed message.

PROGRAM

Sender Program

The sender converts the input text to binary, encodes each 4-bit group with Hamming (7,4), writes the encoded stream to `channel.txt`, and displays the binary and encoded data.

```
def hamming_encode(data):
    result = ""
    for i in range(0, len(data), 4):
        d = data[i:i+4]
        while len(d) < 4:
            d += "0"
        d1, d2, d3, d4 = map(int, d)
        p1 = d1 ^ d2 ^ d4
        p2 = d1 ^ d3 ^ d4
```

```
        p4 = d2 ^ d3 ^ d4
        code = [p1, p2, d1, p4, d2, d3, d4]
        result += ''.join(map(str, code))
    return result

text = input("Enter text: ")
binary = ''.join(format(ord(ch), '08b') for ch in text)
encoded = hamming_encode(binary)
with open("channel.txt", "w") as file:
    file.write(encoded)
print("Binary Data :", binary)
print("Hamming Code:", encoded)
print("Data saved in channel.txt")
```

Receiver Program

The receiver checks each 7-bit codeword, calculates the syndrome, corrects a detected single-bit error, extracts the data bits, and converts them back into text.

```
def hamming_decode(code):
    data = ""
    for i in range(0, len(code), 7):
        block = list(map(int, code[i:i+7]))
        if len(block) < 7:
            continue
        s1 = block[0] ^ block[2] ^ block[4] ^ block[6]
        s2 = block[1] ^ block[2] ^ block[5] ^ block[6]
        s4 = block[3] ^ block[4] ^ block[5] ^ block[6]
        error_position = s1 + (s2 * 2) + (s4 * 4)
        if error_position != 0:
            print("Error detected at position:", error_position)
            block[error_position - 1] ^= 1
            print("Error corrected.")
        data += str(block[2]) + str(block[4])
        data += str(block[5]) + str(block[6])
    return data

with open("channel.txt", "r") as file:
    encoded = file.read().strip()
binary = hamming_decode(encoded)
text = ""
for i in range(0, len(binary), 8):
    byte = binary[i:i+8]
    if len(byte) == 8:
        text += chr(int(byte, 2))
print("Received Data:", text)
```

TEST RUN

Input: HELLO

Sender Output:

Binary Data: 0100100001000101010011000100110001001111

Hamming Code: <encoded binary data>

Data saved in channel.txt

Receiver Output:

Received Data: HELLO

If a single bit in `channel.txt` is changed manually, the receiver should identify the error position, correct the bit, and reconstruct the message. The supplied record gives position 5 as an example.

RESULT

Thus, Hamming Code was implemented to detect and correct a single-bit error in the transmitted data.